

Taller guiado: aplicación React para login y gestión de cursos

useState, useEffect, useContext, React Router, Material UI y localStorage

Objetivo del taller

Construir una SPA sencilla en React que consuma una API REST para:

- iniciar sesión con usuario y contraseña;
- guardar el token JWT en `localStorage`;
- proteger rutas usando `useContext`;
- listar cursos desde el backend;
- crear cursos mediante un formulario con Material UI.

Alcance

Solo se implementan las pantallas de **login**, **lista de cursos** y **agregar curso**.

Endpoints que usaremos

Login

POST localhost:8080/auth/api/auth/login

Listar cursos

GET localhost:8080/auth/api/courses

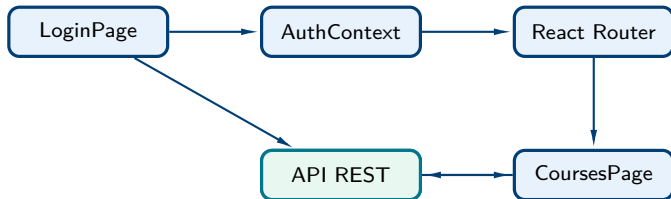
Crear curso

POST localhost:8080/auth/api/courses

Autenticación

Después del login, las peticiones de cursos deben enviar el encabezado Authorization: Bearer TOKEN.

Arquitectura de la aplicación



- AuthContext: estado global de autenticación.
- localStorage: persistencia del token al recargar.
- ProtectedRoute: evita entrar a cursos sin login.

Paso 1: Crear el proyecto

```
1 npm create vite@latest cursos-app -- --template react
2 cd cursos-app
3 npm install
4 npm install @mui/material @emotion/react @emotion/styled @mui/icons-material react-router-dom
5 npm run dev
```

Verificación

Abrir la URL indicada por Vite y confirmar que la aplicación React carga correctamente.

Paso 2: Estructura sugerida

```
1 src/  
2   api/  
3     client.js  
4   auth/  
5     AuthContext.jsx  
6     ProtectedRoute.jsx  
7   pages/  
8     LoginPage.jsx  
9     CoursesPage.jsx  
10  App.jsx  
11  main.jsx
```

Idea

Separar API, autenticación y páginas evita que App.jsx crezca demasiado.

Paso 3: Crear src/api/client.js

```
1 const API_URL = 'http://localhost:8080/auth';
2
3 export async function request(path, options = {}) {
4   const token = localStorage.getItem('token');
5
6   const response = await fetch(`${API_URL}${path}`, {
7     ...options,
8     headers: {
9       'Content-Type': 'application/json',
10      ...(token ? { Authorization: `Bearer ${token}` } : {}),
11      ...options.headers,
12    },
13  });
14
15  if (!response.ok) {
16    const message = await response.text();
17    throw new Error(message || 'Error en la petición');
18  }
19
20  if (response.status === 204) return null;
21  return response.json();
22 }
```

Preguntas guía: cliente HTTP

- ❶ ¿Por qué conviene tener una función `request` reutilizable?
- ❷ ¿Dónde se agrega el token JWT a cada petición?
- ❸ ¿Qué pasaría si el backend responde con error 401?
- ❹ ¿Por qué usamos `localStorage.getItem('token')` dentro del cliente?